

# Depuración de programas

Esteban Álvarez

# Depuración

- Cuando encontramos **errores de ejecución** en nuestro programa o el **resultado no es el esperado** podemos depurar para solucionarlos
- Existen dos métodos:
  - a. `System.out.println()`
  - b. Utilizar el depurador

# Depuración

## PRINTLN

- Copia el siguiente código y ejecútalo
- Comprueba si muestra el mensaje “i vale 49”

```
for (int i=0;i<100;i+=2) {  
    if (i==49) {  
        System.out.println("i vale 49");  
    }  
}
```

- Utiliza println para solucionar el problema

# Depuración

## PRINTLN

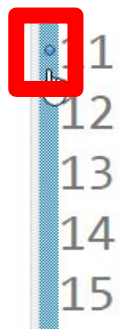
```
for (int i=0;i<100;i+=2) {  
    //Mostramos el valor de i para ver qué pasa  
    System.out.println(i);  
    if (i==49) {  
        System.out.println("i vale 49");  
    }  
}
```

- Mostrando el valor de i comprobamos que nunca vale 49

# Depuración

## DEPURADOR

- El depurador permite hacer una **ejecución paso a paso** del programa y ver el valor que toman las variables
- El primer paso es poner un **breakpoint**

A screenshot of a code editor with a light blue background. On the left, there is a vertical line of numbers 1 through 15, representing line numbers. A red square highlights the number 1. To the right of the line numbers, there is a code snippet in Java. The code is: 

```
for (int i=0;i<100;i+=2) {  
    if (i==49) {  
        System.out.println("i vale 49");  
    }  
}
```

- Será el punto en el que se detenga el programa cuando depuremos
- Puede haber más de un breakpoint

# Depuración

## DEPURADOR

- Después hay que **lanzar una depuración**
- La ejecución se detiene en la línea del breakpoint y la interfaz de Eclipse cambia a “vista de depuración”
- Se puede alternar entre las vistas “java” y “depuración” en la parte superior derecha:



# Depuración

## DEPURADOR

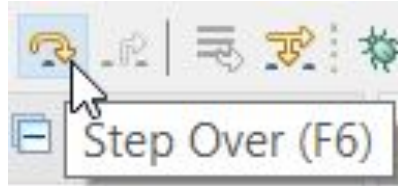
- La ejecución se detiene en la línea del breakpoint
- La línea se marca en color verde y todavía no se ha ejecutado

```
11  for (int i=0;i<100;i+=2) {  
12      if (i==49) {  
13          System.out.println("i vale 49");  
14      }  
15  }
```

# Depuración

## DEPURADOR

- Hay varias formas de ejecutar la línea. Por ahora, saber que la más habitual es “step over”



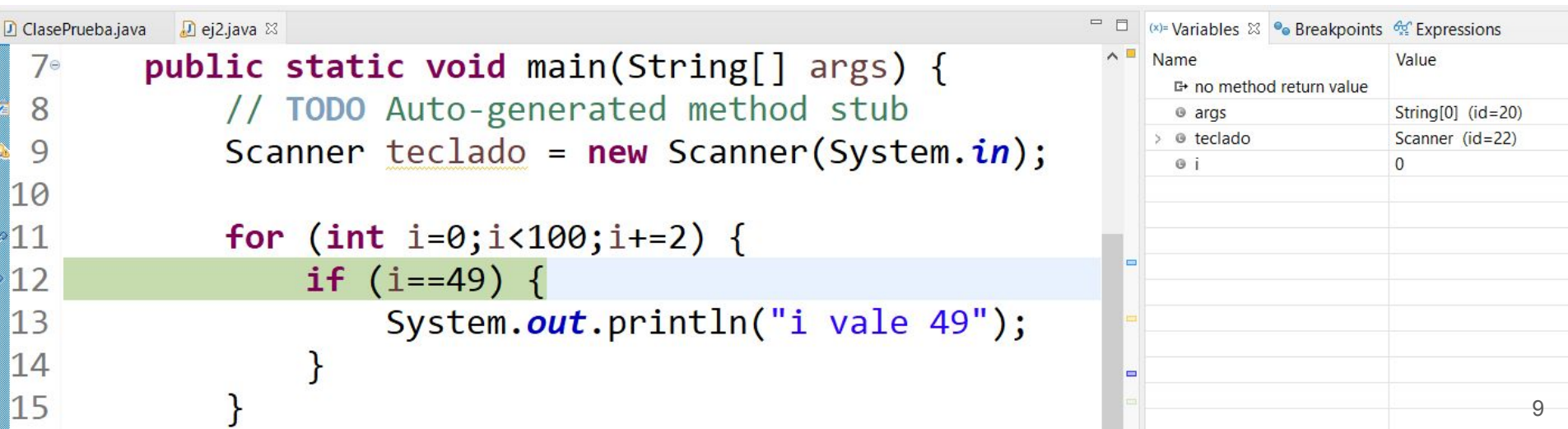
- “Step over” ejecuta la línea actual



# Depuración

## DEPURADOR

- La línea se ejecuta y se pasa a la siguiente. También se puede ver en el panel de la derecha el valor de las variables



The screenshot shows an IDE with a Java file named `ej2.java` open. The code is as follows:

```
7 public static void main(String[] args) {  
8     // TODO Auto-generated method stub  
9     Scanner teclado = new Scanner(System.in);  
10  
11     for (int i=0;i<100;i+=2) {  
12         if (i==49) {  
13             System.out.println("i vale 49");  
14         }  
15     }
```

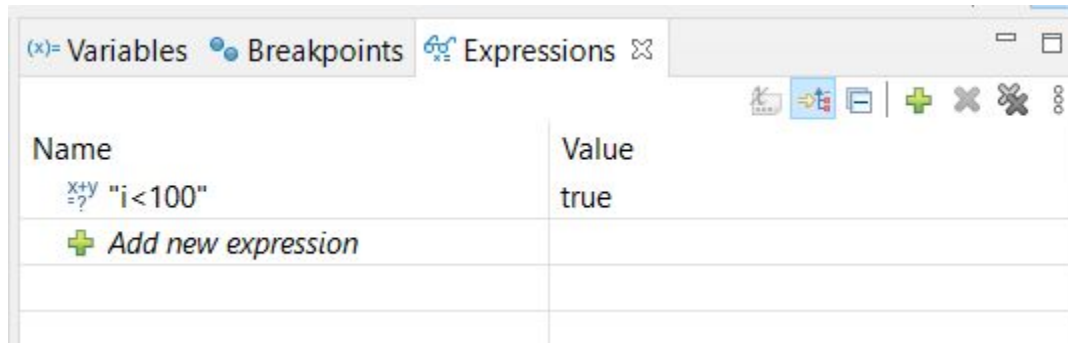
The line `if (i==49) {` is highlighted in green, indicating it is the current execution point. To the right, the 'Variables' panel is visible, showing the current state of variables:

| Name                   | Value             |
|------------------------|-------------------|
| no method return value |                   |
| args                   | String[0] (id=20) |
| teclado                | Scanner (id=22)   |
| i                      | 0                 |

# Depuración

## DEPURADOR

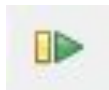
- Podemos añadir expresiones para evaluar su valor en tiempo de ejecución:



# Depuración

## DEPURADOR

- Cuando ya no nos interese depurar en ese breakpoint y queramos **pasar al siguiente punto de ruptura**, pulsamos la tecla “resume”



- El botón de “parar” finaliza totalmente la depuración



# Depuración

**IMPORTANTE:** Es imposible ser un buen programador si no se domina la depuración de aplicaciones

# Depuración

## DEPURADOR

- Vídeos sobre depuración
  - a. <https://www.youtube.com/watch?v=ymV7IUUHkUU>
  - b. <https://www.youtube.com/watch?v=qvNEQ2nAEVE>

**FIN!**